

Cache Write Policies

A cache sits between a client and a slower backing store. The policy that governs how writes reach the store determines both performance and durability of cached data under failure.

Write-through caching writes to the cache and the backing store at the same time. After a successful write, the two never diverge. Reads after a write always see the same value in both places. The cost is latency: every write pays for the store round trip before the client continues.

Write-back caching writes only to the cache and marks the entry dirty. The store write is deferred until the entry is evicted or flushed. This is faster under write-heavy load, but a crash before the flush can lose pending writes that only existed in memory.

Trade-offs and failure modes

Cache stampede happens when many clients miss the same key at once and all recompute the value. Mitigations include single-flight locks, probabilistic early expiration, and staggered TTLs so clients do not expire together.

For a payment ledger, prefer write-through because losing a write is unacceptable. For a page view counter, write-back is acceptable because a few lost increments do not matter to the product.

Read-through caches populate themselves on miss. Write-around skips the cache on write and only fills it on subsequent reads, which can reduce cache pollution for write-once data that is rarely read again.

Operational guidance

Document the durability expectation of each cached value before picking a policy. Measure p99 write latency under load when evaluating write-through. Measure dirty-entry exposure windows when evaluating write-back.

Warm the cache after deploy when cold misses would stampede. Prefer explicit flush points before planned process restarts when dirty write-back entries exist.

Summary: write-through gives strong consistency at higher write latency. Write-back gives lower write latency with durability risk until flush. Stampede is many concurrent misses for one key; protect it with coordination.